Design layer

the whole layer, as one document

Contents

1 Durable jobs 4

1.1 Foundation 4
1.2 Pending updates 5
1.3 System at a glance 5
1.4 Whole model 7
1.5 Worker definitions and codecs 10
1.5.1 Public modules and defaults 11
1.6 Admission and uniqueness 12
1.6.1 Conservative uniqueness profile 13
1.6.2 Caller-owned business transaction 13
1.7 Storage and migrations 13
1.7.1 Installation isolation 14
1.7.2 Migration execution 14
1.7.3 Adding a migration version 15
1.8 Attempts and acknowledgements 15
1.9 Cancellation and operator recovery 17
1.10 Runtime capacity and deadlines 19
1.11 Reads, administration and retention 20
1.12 Observations 23
1.13 Deployment obligations 25
1.14 Verification and retained extensions 25
1.14.1 Complete capability inventory 26
1.15 Verification method 27
1.15.1 Validation profiles 27
1.15.2 Qualification evidence 28
1.15.3 Benchmark method 28
1.15.4 Resilience harness 28
1.15.5 Oracle method 30
1.15.6 Usage examples 30

Glossary 30

Durable jobs

1 Durable jobs

1.1 Foundation

Grind admits typed work durably and fences each execution attempt. Applications own the meaning and safety of external effects.

GOAL

Admit typed work without executing it

An application defines a Worker, starts one runtime, and submits a Job with immediate or delayed availability. The durable receipt and typed handle support later recovery.

GOAL

Preserve uncertainty across process and database failures

Lost replies require reconciliation against durable commands. Expired ownership cannot acknowledge work. Safe delivery after an unknown effect requires an explicit application replay policy or an operator decision.

GOAL

Retain the full job-system scope

Ordinary queues, uniqueness, retries, schedules, administration, observations and testing compose independently. Broader queue controls, dependency workflows, batches and other retained capabilities remain specified below with explicit gaps.

NO-GOAL

Guarantee exactly-once external effects

PostgreSQL transitions are atomic. An external payment, message or HTTP call is outside that transaction unless the application supplies its own durable protocol.

NO-GOAL

Own application authentication or compensation

Operator attribution is evidence, not authorization. Saga owns typed workflow progress and compensation; a Grind job invoking a local Saga does not make each Saga step durable.

INVARIANT

Admission and execution are separate

Input and key encoding precede storage. Admission never invokes the handler. Exact deployed worker and codec versions are required to execute stored input.

mechanism

INVARIANT

A retry counter never grants ownership

All execution writes require the complete Attempt fence. Refunding an Attempt ordinal does not restore an expired epoch.

mechanism

INVARIANT

Runtime failure never invents a business error

Typed errors retain their codec or remain explicitly unrecorded. Operational failure, contract mismatch and uncertainty have separate representations.

mechanism

PRINCIPLE

Use durable rows as authority

An event, a returned handler value, or an elapsed timeout cannot establish a committed job outcome. Recovery reads the matching durable receipt or current job.

PRINCIPLE

Make resource ownership explicit

Grind owns its supervised processes, main pool lifetime and reserved renewal pools. Applications may borrow the main pool and own outer business transactions.

- The scope boundary and the oracle boundary are fixed in ADR-0001. The source provenance of the implementation is fixed in ADR-0011.

1.2 Pending updates

ruling **Define a hard bound for queued storage checkout** 2026-10-06

The absolute deadline is calculated before pgo checkout, but the pinned pool does not arm a receive timeout while queued. Pool overload shedding or connection transfer ends that wait. Decide whether the public deadline must include a hard queue-wait bound.

ADR-0008

build **Complete retained queue and schedule capabilities** 2026-10-06

Priority, live controls, cluster/rate/partition limits, recurring schedules, notification infrastructure, plugin contracts and richer administration need executable contracts and implementations. Local concurrency and delayed admission implement narrower behavior.

ADR-0001

build **Complete dependency workflows, batches and chunks** 2026-10-06

Durable dependency edges, named step results, batch callbacks and homogeneous chunk dispatch retain separate contracts. They must preserve admission ambiguity, attempt fencing and per-item outcomes without importing Saga compensation into the queue core.

ADR-0010

build **Extend uniqueness beyond the conservative profile** 2026-10-06

Cross-worker matching, arbitrary state sets, broader replacement, backfill and bulk uniqueness need concrete equality, locking and reconciliation contracts. Ordinary bulk insertion cannot imply unique admission parity.

ADR-0003

verify **Establish fresh qualification for the chosen release** 2026-10-06

No qualification run exists for the chosen release. A qualification must record source, dependencies, hardware, budgets, validity verdicts and failed runs; existing scenario coverage does not prove unconditional liveness or full Oban/Pro parity.

ADR-0009

ruling **Decide fallback installation identity strength** 2026-10-06

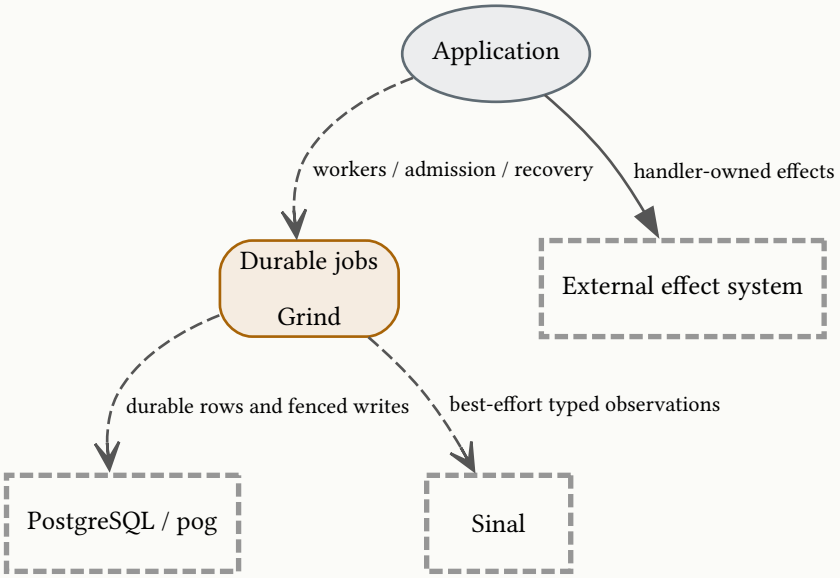
When either endpoint lacks a PostgreSQL cluster identifier, binding compares only database OID and schema. Distinct clusters may collide and compatibility is nontransitive. A stricter identity or constrained deployment contract needs an explicit decision.

ADR-0008

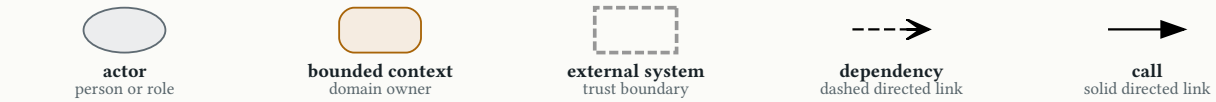
1.3 System at a glance

One Durable jobs context owns admission, persisted outcomes, attempt authority and queue runtime.

VIEWPOINT CONTEXT-OWNERSHIP · ALTITUDE L1 · BOUNDARY Application and queue ownership

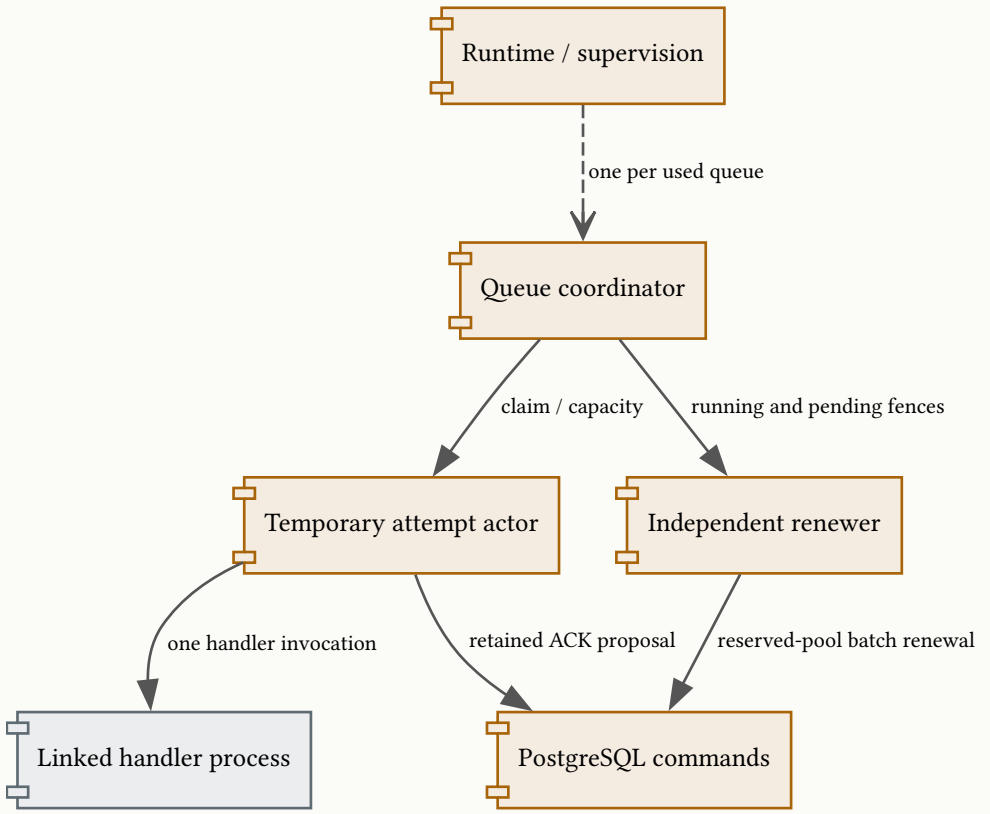


LEGEND



Dependency arrows do not confer authorization or shared transaction ownership.

- The runtime targets Erlang/OTP. Direct ecosystem dependency is Sinal. Standard Gleam JSON supplies ordinary codecs; schema-aware conversion is an application choice.
- The main pool is configured by the caller's pog.Config. Each used queue adds one consumer, a renewer and a reserved one-connection renewal pool. Pruning is optional; processing can be disabled while admission and reads remain available.



The coordinator keeps capacity occupied until an attempt and its acknowledgement settle. Renewal has an independent actor and pool.

- Local units are Worker definitions, Admission and uniqueness, Storage and migrations, Attempts and acknowledgements, Cancellation and recovery, Runtime capacity, Reads and retention, Observations, and Verification and extensions.

1.4 Whole model

Persistent job identity, command identity and execution authority have different lifetimes.

Job state queued Ready ordinary work. scheduled Delayed work awaiting availability. retryable Observed failure scheduled for another attempt. executing Owned by a live attempt fence. succeeded Committed typed output. business_failed Terminal business failure or scheduling budget exhaustion. runtime_failed Terminal operational failure. contract_mismatch Stored/deployed contract refused. uncertain Effect outcome requires investigation; nonterminal. discarded Explicitly discarded with reason. cancelled Committed cancellation; not proof of absent effects.

Job

entity stateful Durable jobs owned by Durable jobs

Retained record of a submitted versioned worker contract.

ATTRIBUTES

derived **Identity** · Installation × Int
Database-generated job ID is installation-local and never substitutes for a submission key.

authored **Worker contract** · Worker ID × worker version × codec versions
Input/output and optional error versions are checked at execution and read attachment.

authored **Scheduling** · Queue × availability × limits
Relative availability is resolved by PostgreSQL. Attempts, snoozes and replays have separate counters.

derived **State** · Job state job-state · job-lifecycle
Only committed storage commands change lifecycle state.

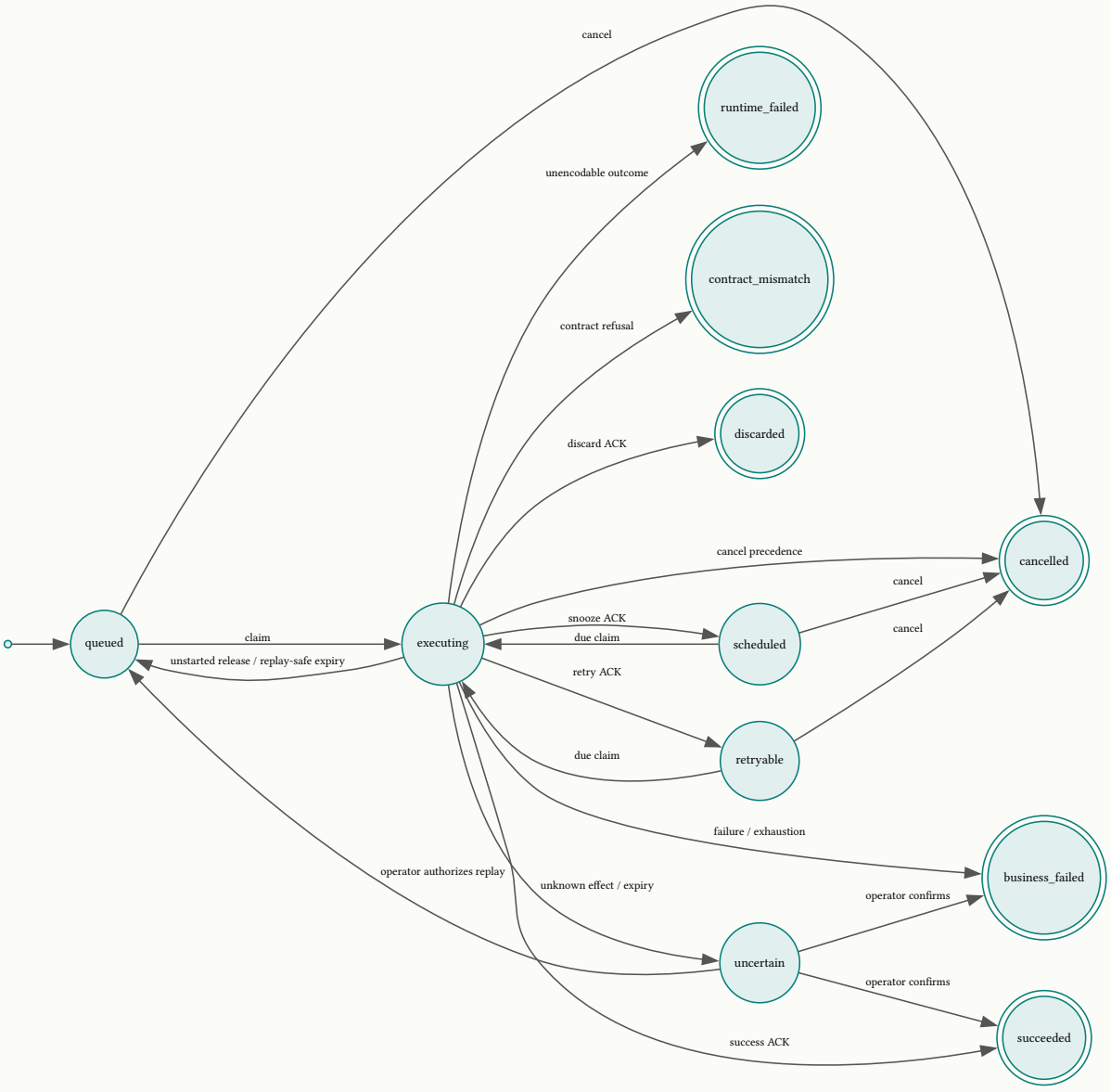
derived **Execution evidence** · Optional attempt fence / cancellation / uncertainty
Uncertain retains effect evidence and ownership evidence. Cancellation intent remains independent until attributed terminal resolution settles the job.

derived **Outcome and retention** · Optional encoded output/error/reason/cause/finished time
finished_at exists for six terminal states and never for Uncertain.

RELATIONSHIPS

`1 : 0..n` Admission, acknowledgement and operator-resolution receipts refer to the job and cascade when it is pruned.

STATE MACHINE



- Immediate admission chooses Queued; delayed admission chooses Scheduled. The single diagram initial node represents ordinary admission. Eligibility also requires queue, exact worker version, due database time, remaining budget and current row locks; an arrow alone grants no execution authority.

Admission receipt

entity · immutable · Durable jobs · owned by Admission

Original command decision retained independently of later job state.

ATTRIBUTES

authored **Identity** · Installation × submission key
All submissions retain a supplied or generated identity.

derived **Fingerprint** · SHA256 prepared command
Plain and unique commands use different domain tags. Exact encoded input, worker/codecs, queue, limit, schedule and uniqueness action participate; correlation and maximum replay count do not.

derived **Decision** · Inserted | Existing | Rescheduled
Original job, actual conflict queue/state and decision time survive matching-window and job-state changes.

Attempt fence

value-object · immutable · Durable jobs · owned by Attempt execution

Authority to write one executing attempt.

ATTRIBUTES

derived **Identity** · Job ID × attempt ID × epoch × owner
Attempt ID and epoch distinguish claims even when the retry ordinal repeats. Owner includes runtime incarnation.

derived **Validity** · Executing state × live database lease × contract match
Exact lease expiry is invalid. Renewing the same owner changes lease expiry without changing the authority identity.

Uniqueness policy

value-object · immutable · Durable jobs · owned by Admission

Typed candidate selection and conservative conflict action.

ATTRIBUTES

authored **Key and scope** · Full input | selected key; WithinQueue | AcrossQueues
Selected keys retain a nonempty name/version and typed encoder. Worker identity/version always participate.

authored **Period** · Within(positive duration, origin) | WhileRetained
Origin is insertion or schedule; supported millisecond bounds are checked by the constructor.

authored **Eligible states** · Incomplete | ScheduledOnly | IncompleteOrSucceeded | AllRetained
These profiles filter current rows without changing key identity.

authored **Action** · KeepExisting | RescheduleTo(timestamp)
Only a scheduled candidate may change availability. Input, worker/version and identity remain unchanged.

Refinement	Validation owner	Failure contract
Worker/codec/queue identity and positive source limits	Source-owned builders	Panic at definition time; no deployment starts
Runtime schema, deadlines, used queue tuning, duplicate workers	Config.check and start	Typed configuration error before resources start
Encoded input/key and payload byte limit	Preparation	InvalidInput before checkout or writes

Refinement	Validation owner	Failure contract
Output/error codec acceptance and byte limit	After handler returns	Terminal runtime failure; no synthetic business error
Nonempty resolution identity/by/details	Operator command construction	Rejected attribution/command before resolution
List limit and quarantine/prune batch	Administration	Positive bounded page/batch; maximum ten thousand

Axis	Owner and identity	What it proves
Submission	Caller/generated key and exact command	One admission decision, while receipt remains
Uniqueness	Worker/key contract and incoming eligibility	An equivalent retained candidate was selected
Execution	Attempt fence and database time	Authority for a current storage write
Acknowledgement	Command ID and exact proposal/fence	The actual committed disposition
Resolution	Resolution ID, decision and attribution	An operator command was applied
Correlation	Application tracing value	No authority or idempotency guarantee

- The typed JobHandle retains installation, queue, worker contract and output/error codecs. A Conflict includes its real route and compatible handle. PendingSubmission retains the prepared fingerprint; it is not a handle proving admission.

1.5 Worker definitions and codecs

Typed application values cross storage through explicit versioned JSON contracts.

Worker definition

responsibility

Retain typed handlers and all policies needed to interpret stored input.

interface

worker.new wraps Result; worker.responding receives Context and returns Response. Builders select queue, version, codecs, attempts, timeout, snoozes, retry and abandonment.

interactions

Configuration erases heterogeneous workers through retained typed closures. Storage and reads check declared versions before decoding.

invariants

Duplicate worker ID/version is rejected across the runtime. Multiple versions may coexist; there is no fallback. Source-owned invalid constructors panic; runtime Config.check returns typed validation errors.

failure

Invalid input/key encoding refuses admission before storage. Stored contract mismatch prevents handler execution. Output/error refusal after effects becomes terminal RuntimeFailed, with no retry.

Response	Scheduling/accounting	Outcome ownership
Succeeded(output)	Terminal; output encoded once	Output codec refusal is operational failure
Failed(error)	Retry only below limit and when policy permits	Optional error codec preserves typed error; otherwise explicitly unrecorded
Snoozed(delay, reason)	Refund attempt; increment separate snooze count	Snooze exhaustion is business failure with cause
Discarded / Cancelled	Terminal reason	Does not prove earlier effects absent
Uncertain(evidence)	Bypasses retry policy	Operator recovery or explicit replay safety required

- Context retains job/attempt/queue, correlation, attempts/snoozes, optional main-pool connection and a handler-owned cooperative cancellation selector. A forged local testing context grants no storage authority.

BEHAVIOR

An encoding failure follows an executed effect

Given The handler returned a value after executing application code.

When The output or configured error encoder rejects it or exceeds the payload limit.

Then The acknowledgement proposes RuntimeFailed and does not request another attempt. The failure does not undo effects.

level: boundary area: Worker outcomes

- The default payload bound is one MiB. No codec version string proves equivalent implementations; applications change versions and retain compatible deployed code. See ADR-0002, worker contracts and external codec consumer.

1.5.1 Public modules and defaults

Module	Holds
grind	Config, the runtime (start, supervised, named, stop, connection, migrate), submit, submit_in, reads, await, cancel, errors and their kinds
grind/worker	Workers, codecs, Response, retry, timeout, snooze and abandonment policies, handler Context
grind/job	Job builder (new, with_id, at, after, unique, with_correlation), JobHandle, State, TerminalCause
grind/queue	Per-queue concurrency, polling, lease and shutdown grace
grind/unique	Uniqueness policies
grind/admin	list, statistics, resolve_uncertain, resolve_uncertain_in, quarantine_expired, prune_finished, reconcile_acknowledgement
grind/telemetry	Sinal event descriptors
grind/testing	perform, perform_with, drain

- Modules under grind/internal carry no stability contract. Error and outcome types may gain variants, so a caller that must not break on a new variant branches on submit_error_kind, read_error_kind or cancel_error_kind. The describe functions render one log line.

Setting	Default	Changed by
Handler execution timeout	15 minutes, then the abandonment policy applies	worker.with_timeout
Snoozes per job	100, then Failed with SnoozeLimitReached	worker.with_max_snoozes
Business attempts	20	worker.with_max_attempts, job.with_max_attempts
Retry delay	15 seconds doubling to one day, plus up to ten percent jitter, clamped to supported nonnegative bounds	worker.with_retry_policy
Abandoned attempt	HoldUncertain	worker.with_abandonment
Job retention	pruner on, finished jobs deleted after seven days	grind.with_pruner, grind.without_pruner
Encoded input, output and error size	1 MiB, then PayloadTooLarge	grind.with_max_payload_bytes
Initial connection at start	15 seconds, then Unavailable	grind.with_connect_timeout
Storage call deadline	4 seconds absolute; queued checkout may exceed it	grind.with_statement_deadline
Uniqueness lock wait	2 seconds, then UniquenessContended	grind.with_unique_lock_wait
Migration step deadline	30 seconds	grind.with_migration_deadline
Queue concurrency per node	10	queue.with_concurrency
Poll interval	250 milliseconds	queue.with_poll_interval
Attempt lease	30 seconds, renewed every 10 seconds	queue.with_lease
Shutdown grace	15 seconds	queue.with_shutdown_grace

Setting	Default	Changed by
Observation forwarder	1024 events in flight, then dropped and counted	grind.with_observation_capacity
Worker and codec versions	1	worker.with_version, worker.with_codec_version
Queue	default	worker.with_queue, job.with_queue

Rejected value	Result	Write
Input or selected unique key at submission	InvalidInput with the reason, before any connection is used	None
Handler output or error after the handler ran	Terminal RuntimeFailed with the output codec reason; no retry	Acknowledgement
Value confirmed by an operator resolution	ResolutionValueRejected with the reason	None

- A validating codec encoder returns Result with a reason, and worker.infallible wraps a plain encoder. A json_blueprint codec maps its encode error to the reason with its describe function.

1.6 Admission and uniqueness

One atomic admission command combines receipt lookup, candidate selection and any permitted mutation.

Admission

responsibility

Prepare exact typed input and durably choose Inserted, Existing or Rescheduled.

interface

job.new and setters; grind.submit; submit_in; reconcile_submission; Admission and SubmitError.

interactions

Prepares input/key before checkout; PostgreSQL locks the receipt/domain and candidate rows; typed handles retain the selected actual route.

invariants

All submissions have receipt identities. Matching receipt is checked before a new decision. Same identity/different fingerprint is IdConflict. Admission never runs the handler.

failure

InvalidInput and known pre-commit refusal differ from CommitUnknown. An absent receipt during reconciliation remains unknown; it is not proof that the original transaction cannot still commit.

- with_id supplies a submission key, not the durable integer job ID. Retrying ordinary job.new without preserving a key creates another command. A PendingSubmission is the preferred retained recovery token after CommitUnknown.
- after resolves its duration against database time at admission. at retains an absolute timestamp, with past values made immediately eligible. Correlation and replay policy are retained on insertion but excluded from receipt equality; changing either under an existing key returns the original decision.

BEHAVIOR

A lost admission reply is reconciled

Given A prepared command returned CommitUnknown and its original PendingSubmission remains available.

When Reconciliation finds its matching durable receipt.

Then It returns the original admission decision without rerunning uniqueness or executing work. A conflicting receipt returns IdConflict; missing receipt remains unknown.

level: boundary area: Admission

1.6.1 Conservative uniqueness profile

- full_input uses input JSON and its codec version. selected uses a named key, key codec version and typed projection. Jobs without matching retained key material do not participate; arbitrary ordinary rows cannot be reconstructed from a new projection closure.
- Equality hashes PostgreSQL jsonb text: object order is normalized; repeated fields use PostgreSQL last-wins semantics; array order matters; exponent notation normalizes; 1 and 1.0 remain distinct. This differs from exact prepared-request byte equality and from Oban Basic containment.
- Worker ID/version always scope the domain. WithinQueue is default; AcrossQueues stays inside the installation. Defaults are Incomplete states and KeepExisting. Incomplete includes Uncertain; other profiles are ScheduledOnly, IncompleteOrSucceeded and AllRetained.
- within uses a positive bounded duration from insertion or scheduled time with an inclusive cutoff. while_retained has no time cutoff; pruning ends participation. Incoming period and state filter are predicates, not extra namespaces.
- A transaction advisory lock hashes schema, worker ID/version, key contract and PostgreSQL key digest; queue is excluded, so same-key admission serializes even across queue-local policies; FOR KEY SHARE protects candidates against pruning. The default lock wait is two seconds and must clear the storage deadline by at least one second. Lock timeout is UniquenessContended, not Existing.
- Reschedule changes availability only on eligible scheduled work, preserving identity, payload and contract. Other states return Existing. Across-queue conflict handles retain the selected queue, not the submitting queue.

1.6.2 Caller-owned business transaction

- submit_in accepts a checked SingleConnection inside an open READ COMMITTED transaction for the same database. It issues no BEGIN or COMMIT, temporarily scopes search_path and lock_timeout, and restores them. A failed statement may abort the caller transaction.
- Its reply means staged admission, not commit. No lifecycle admitted event is emitted. The borrowed call inherits the outer transaction timeout rather than acquiring another Grind deadline. SERIALIZABLE and REPEATABLE READ are refused without weakening the caller transaction. Applications retaining those isolation levels may commit a business outbox and dispatch it separately under stable submission identity.
- After a lost outer COMMIT reply, reconciling an admission receipt proves that job admission occurred. An older matching receipt cannot prove that this retry's surrounding business writes committed. The application needs its own business-command reconciliation.
- Concrete authority and alternatives are recorded in ADR-0003 and ADR-0004. Executable consumers: admission and concurrent uniqueness tests.

1.7 Storage and migrations

The adapter owns one exact schema and forward-only transactions. Unknown commit does not authorize blind repetition of effects.

PostgreSQL adapter

responsibility

Scope SQL to one installation, preserve physical schema contracts and distinguish known refusal from uncertain reply.

interface

grind.Config from pog.Config; start/migrate/connection; internal bounded checkout and generated or dynamic queries.

interactions

Main pool is shared with application queries. Borrowed admission uses caller transaction settings; migrations use independent step budgets.

invariants

Grind-owned transactions pin READ COMMITTED and restore application search_path. A failed cleanup retires the connection. The bounded wrapper runs its callback at most once and never retries its SQL body after stale-holder replacement.

failure

Unavailable pools, connection loss and uncertain COMMIT can deny a reliable reply. Deadline and cleanup cannot prove rollback. Installation binding is a misuse guard, not authentication.

- Client installation identity uses database OID, schema and optional pg_control_system cluster ID. Cluster comparison is enforced when both values exist; fallback OID/schema comparison has collision and nontransitivity limits. Schema identifiers are quoted and validated for PostgreSQL length and reserved/unsafe names.
- Existing schema lookup avoids demanding database CREATE permission from an installed role. Concurrent creation is rechecked. The original creation error is retained if that recheck fails.
- Source migrations are versions 11 to 13. Version 11 is the baseline; 12 adds finished-time retention and dependent receipt lifecycle; 13 adds correlation and replay accounting, uncertainty indexing and snooze cause. The hand-maintained source list is authoritative. priv migrations mirror exact step SQL for Cigogne consumers.
- Each step takes a schema-keyed transaction advisory lock first, rereads markers, applies DDL, writes the marker and checks physical shape before COMMIT. Earlier committed steps survive a later failure. Unknown step reply can be safely rechecked/rerun because lock and marker decide application.
- Shape validation rejects unsupported marker versions, wrong relations/columns/indexes/sequences/foreign keys, obsolete storage_owner and stray Grind-prefixed objects. It does not quietly adopt an Oban schema. Experimental version 10 needs an explicit reinstall; no automatic destructive reset is performed.
- Default migration budget is thirty seconds per step and lock wait two seconds. Large table work can exhaust it; default finalization and foreign-key validation acquire real locks. Operators quiesce/drain and budget migration rather than infer a zero-downtime guarantee.
- Static SQL is fixed at compile time, lives in one file per query and is generated by Squirrel; the generated module is replaced wholesale and never edited. Dynamic SQL stays hand-written: shared lease, period and lock predicate fragments, per-disposition acknowledgement SQL, nullable-parameter queries and candidate selection. The split is permanent, see ADR-0014, ADR-0007, ADR-0008 and migration mirrors.

1.7.1 Installation isolation

- One installation is one PostgreSQL schema. Every job, quarantine scan, uniqueness domain and retention sweep covers exactly the tables of that schema, and no owner column scopes rows inside a shared schema. Roles hold the intended schema privileges, and a pool is invalid after the runtime stops.
- The schema is explicit configuration: grind.new selects public and with_schema overrides it. Each storage call sets search_path to exactly that schema, quoted, and restores the caller value before the connection returns to the pool, so application queries on the shared pool keep the application search_path. The uniqueness advisory lock binds the configured schema as a parameter and never resolves it on the server. The rationale is recorded in ADR-0017.
- Pools configured with the same schema in one physical database share one installation. Pools configured with different schemas are isolated, because each schema holds a complete table set. Two installations in one database need distinct schema names, and nothing enforces that.
- grind.migrate creates the configured schema only after confirming its absence, because CREATE SCHEMA IF NOT EXISTS demands database-level CREATE privilege even for an existing schema. start and every other call never create a schema and return a typed storage error against missing tables.
- The least-privilege setup has an administrator create the schema once, owned by the connecting role, and pass its name to with_schema; the role then needs no database-level CREATE.
- A connection pooler must preserve search_path and keep one server session for the whole checkout. Transaction pooling can attach a client to a connection without its startup parameters and silently point an installation at another schema. Poolers and encrypted fault paths need deployment qualification.

1.7.2 Migration execution

- grind.migrate applies the source migration list in ascending order, one transaction per step. A step pins READ COMMITTED, takes a transaction-scoped advisory lock keyed by a fixed string and the schema, rereads the schema version, skips an applied step, applies its statements including its marker insert, rereads the version and commits. Concurrent callers, including a cigogne run that holds the same lock, serialize on that lock.
- A step sets a transaction-local lock_timeout of two seconds after taking its advisory lock. A statement that cannot acquire its lock in that time fails the step with MigrationLockUnavailable, which is safe to retry. The timeout is set inside grind.migrate only; an application applying the mirror files sets its own.
- A failing statement rolls back its step as MigrationStepFailed and leaves earlier committed steps in place. MigrationCommitUnknown covers a checkout failure before BEGIN, a lost BEGIN reply and a failed ROLLBACK; rerunning is safe in each

case because the reread decides. A foreign, partial, tampered or pre-baseline schema fails closed as IncompatibleSchema or UnsupportedSchemaVersion and is never repaired.

- Each migration is also published under priv/migrations as a cigogne file, named by timestamp and version, with up, down and end markers, plus a cigogne configuration. Its first up statement is the same advisory-lock statement. Grind never reads these files at runtime. One tool owns each schema, either grind.migrate or cigogne, because mixing them can fail on a duplicate object. The down section of version 11 drops every Grind table with all job, receipt and resolution data.
- A cigogne engine driven from Gleam can share the Grind pool by using grind.connection as its connection. Cigogne keeps its own tracking table, by default in the public schema, which never reads the Grind marker table. An application with a non-public Grind schema gives cigogne a connection whose search_path is that schema and places the tracking table in that schema or in a dedicated schema.
- Version 12 performs size-proportional work under one ACCESS EXCLUSIVE lock: the finished_at column and backfill rewrite every row, a check constraint validates every row, and the storage_owner removal rebuilds indexes and primary keys and runs two collision scans. The cost grows with the row count of the jobs table and is bounded by the migration deadline, not by lock_timeout. A table that exceeds the deadline reports MigrationCommitUnknown for version 12 with nothing committed, and every rerun fails identically until the deadline is raised or the file is applied directly outside it.
- Version 12 requires a stop-the-world deployment. Earlier code acknowledging a terminal state writes no finished_at, which the check constraint rejects once version 12 commits; later code writing that column against version 11 hits an undefined column. Every writer stops before migration and the new code deploys afterwards. Version 12 deletes orphan receipt rows before adding each cascading foreign key, so an orphan does not fail the step.
- Operators quiesce and drain consumers before a locking upgrade, inspect the schema generation and physical shape, and budget each step for real DDL and foreign-key validation. Experimental version 10 requires an explicit reinstall. Released mirror SQL is never rewritten. Run grind.migrate as a deployment step, not at node boot, once the jobs table is large.

1.7.3 Adding a migration version

- The source list must be exactly the contiguous range from version 11 to the newest version, so a change adds one new highest version at a time. The mirror file and the source entry move together: the entry statements equal the mirror up section without trailing semicolons, advisory-lock statement included.
- The entry shape declares the complete cumulative set of grind-prefixed relations of that version: every table, sequence and index, including the implicit sequence of a bigserial column and the index behind every primary key and unique constraint. The entry foreign_keys names every foreign-key constraint, checked against the constraint catalog, because a foreign key without its own index never appears in the relation catalog. The schema probe compares the shape exactly, so an unlisted relation fails closed like a missing one; the set is confirmed on a database where the version was applied, not derived from DDL text. See ADR-0015.
- A released mirror file carries a pinned sha256 in the conformance test, so a post-release edit fails; the newest version stays unpinned until it settles. See ADR-0016. The disposable-cluster gate applies every mirror up section with psql before Squirrel generation and check, so a mismatch between the two sources also fails there.

1.8 Attempts and acknowledgements

Each admitted execution obtains a new authority identity. A handler result becomes an outcome only after a fenced command commits.

Claim and attempt execution

responsibility

Claim due work, decode the exact deployed contract, run one handler, and retain its result until settlement.

interface

Internal claim, lease, attempt and acknowledgement commands; testing.drain for explicit manual processing.

interactions

The coordinator claims one slot at a time; a temporary actor owns the linked handler and retained ACK proposal; the renewer maintains live leases.

invariants

Every new write checks executing state, attempt ID, epoch, owner, codec contract and a live database-time lease. Same-owner renewal is not takeover. Handler invocation is not repeated by ACK recovery.

failure

Contract mismatch parks the row before handler execution. Child startup refusal before StartAttempt releases/refunds its exact claim. Handler/attempt death leaves lease-based recovery, not synthetic success or business failure.

Attempt actor phase ready Claim acquired; StartAttempt not yet received. running One linked handler is active. waiting Handler result retained for ACK/reconciliation. finished Completion reported; actor has no further execution authority.

Attempt actor

entity stateful Durable jobs owned by Attempt execution

Temporary local owner of one handler invocation and its acknowledgement proposal.

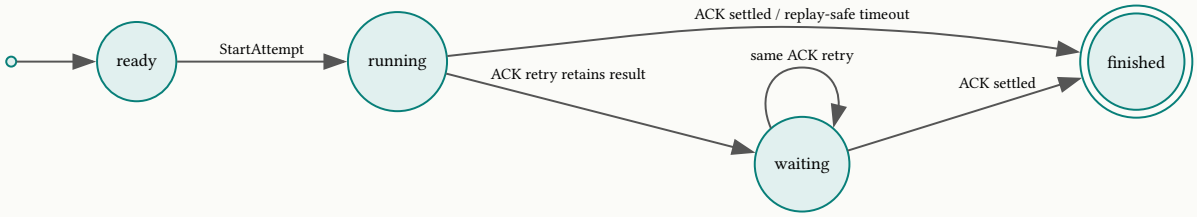
ATTRIBUTES

derived **Request** · ClaimedJob × runtime owner × coordinator/renewer
Retains the attempt fence, deployment and local completion callbacks.

derived **Phase** · Attempt actor phase attempt-phase · attempt-progress
Local phase does not replace the persistent job state.

derived **Pending completion** · Optional Execution × stable ACK command × retry count
Only one handler result is retained. Retry diagnostics count ACK attempts, not business attempts.

STATE MACHINE



- Actor death from linked handler failure is destruction of this local entity, not a committed transition to Finished. The durable job remains fenced until lease recovery. StopWorker destroys the actor from any phase; it does not create a Finished state or resolve storage. A successful first ACK can go directly from Running to Finished.
- Claims increment the attempt ordinal. Snooze refunds one attempt and increments snooze_count. Explicitly replay-safe expiry refunds and increments replay_count, bounded independently. Fresh attempt ID/epoch fence old processes even when the ordinal repeats.
- Due claims select the earliest available_at then job ID among exact deployed worker versions, using FOR NO KEY UPDATE SKIP LOCKED. This is a selection order, not a starvation/fairness guarantee. Each claim poll first quarantines at most one expired row in the queue, including versions absent from the deployed registry. Admin quarantine can cover unpolled queues in bounded batches. HoldUncertain preserves evidence; bounded ReplayAfterLeaseExpiry clears old ownership and queues work only without pending cancellation.
- The handler is linked to the attempt actor. Its finite timeout stops/unlinks it. HoldUncertain proposes uncertainty; explicit replay-safe policy stops renewal and lets lease expiry deliver again. Runtime failure alone is no proof of replay safety.

Acknowledgement recovery

responsibility

Commit one actual disposition and reconcile lost replies without reinvoking the handler.

interface

Stable command ID, exact proposal hash and durable ACK receipt; admin.reconcile_acknowledgement for manual recovery.

interactions

The attempt actor retries both known rollback and ambiguous ACK replies. The consumer keeps its slot occupied; reserved renewal covers running and pending attempts.

invariants

A matching receipt returns the original committed state/cause. Conflicting command/proposal is refused. Receipt lookup is allowed after expiry; a fresh write is not. Cancellation can change a proposed non-uncertain state before commit. An explicit Uncertain proposal preserves its evidence and cancellation intent.

failure

If no receipt is found and ownership has expired, ACK cannot commit. Lease recovery preserves uncertainty. Reconciliation lacks availability detail present only in a fresh UPDATE RETURNING reply.

BEHAVIOR

The first acknowledgement rolls back

Given The handler already returned and the attempt actor retains its proposal.

When The first ACK fails with a known rollback.

Then The actor retries that same command and proposal while its fence permits it. It does not run the handler again and it continues occupying local capacity.

level: boundary area: Attempt recovery

- Completion-pending renewal has a finite lifetime of one lease duration after completion notice. Once that window ends, receipt reconciliation may continue but cannot create a write under an expired fence. A slow ACK row cannot stall healthy siblings because renewal skips locked rows.
- ACK receipts fingerprint the full proposal and fence. Matching receipts are checked before writes and after a zero-row update. Renewing a lease cannot invalidate that command. See ADR-0005 and ACK fault tests.

1.9 Cancellation and operator recovery

A request to stop and evidence of an unknown effect answer different questions.

Cancellation

responsibility

Record inactive cancellation or cooperative executing intent and arbitrate races atomically.

interface

grind.cancel; CancellationResult; worker cancellation selector.

interactions

Committed intent is observed by renewal and delivered to the handler once. ACK checks cancellation under the same fenced row transition.

invariants

Completion first preserves the finished result. Cancellation intent first wins non-uncertain ACK precedence. Explicit Uncertain retains its evidence and cancellation intent in either ordering. Expired ownership cannot overwrite recovery. Repeating an unknown cancellation is safe; Uncertain is not implicitly replayed.

failure

A live handler may ignore intent or already have performed effects. Database/scheduler progress is required for observation; there is no unconditional cancellation latency. Cancellation intent cannot certify that an effect was prevented or completed.

BEHAVIOR

Cancellation precedes an explicit uncertain response

Given Cancellation intent committed while the handler was executing.

When The handler returns Uncertain and its ACK wins with a live fence.

Then Storage commits Uncertain with the exact effect evidence and retained attempt fence. Cancellation intent remains recorded. Operator listing includes the row and ordinary pruning excludes it. AuthorizeReplay is refused. An attributed terminal confirmation can settle the effect.

level: boundary area: Cancellation precedence

BEHAVIOR

Cancellation follows an uncertain acknowledgement

Given An uncertain acknowledgement already committed.

When The caller requests cancellation.

Then The result is AlreadyUncertain. Storage retains the evidence and records cancellation intent. AuthorizeReplay is refused; attributed terminal confirmation remains available.

level: boundary area: Cancellation precedence

- Lease quarantine after pending cancellation also preserves Uncertain and suppresses automatic replay. Matching acknowledgement retries return the original committed Uncertain disposition without reinvoking the handler. ADR-0006 and ADR-0012 record the rationale.

Operator resolution

responsibility

Settle or authorize a new attempt after application investigation of uncertain work.

interface

admin.list; resolution(id, by, details); ConfirmSuccess(output), ConfirmFailure(error), AuthorizeReplay; resolve_uncertain; resolve_uncertain_in returning Staged(Resolved).

interactions

Encodes the selected typed value before storage, locks the uncertain row and commits an attributed durable resolution receipt.

invariants

Nonempty command/attribution fields; exact full command equality; AlreadyApplied returns original state. AuthorizeReplay is refused with pending cancellation; terminal confirmations can settle it. ConfirmFailure requires an error codec.

failure

Value rejection writes nothing. Different reuse is a conflict; lost reply is resolved by repeating the identical command. Authorizing replay cannot certify an external effect did not occur.

Borrowed operator resolution

responsibility

Stage the job transition and its exact attributed receipt alongside caller writes.

interface

admin.resolve_uncertain_in accepts the caller's open READ COMMITTED transaction for the same database and returns Staged(Applied or AlreadyApplied).

interactions

The caller commits the resolution with its application acknowledgment. Grind scopes search_path and caps lock_timeout and statement_timeout by the configured storage deadline while preserving stricter caller limits; successful statements restore all three settings.

invariants

The ordinary typed command validation, receipt equality, attempt fence and cancellation rules remain shared. There is no new transaction owner or committed resolved observation. A prior matching receipt cannot prove the surrounding application's new commit.

failure

A pool or unsupported isolation is refused. A statement error may abort the transaction; callers propagate errors and roll back. The caller owns checkout, network waits and the outer transaction lifetime. Row locks remain until that transaction ends.

BEHAVIOR

Resolution and application acknowledgment share a commit

Given The caller stages an attributed operator resolution together with its application acknowledgment.

When The enclosing transaction commits or rolls back.

Then Both changes become durable together or neither does.

Then A lost commit reply requires durable application readback; the staged result alone proves no commit.

level: boundary area: Operator recovery

- Investigation and external calls precede the borrowed transaction. An application acknowledgment committed with resolution remains available under application retention after job pruning. See ADR-0013 and transactional resolution consumer.
- Applications own operator permissions, evidence gathering, external status queries and replay/idempotency decisions. Resolution IDs and by/details fields do not authenticate anyone. Durable retention bounds command recovery.
- A resolution ID names one exact command across the installation, which is one database and one Grind schema. Reusing an ID for another job or another decision returns ResolutionConflict, so an application qualifies its decision IDs with the job ID. The application persists the decision identity and the full command before applying it, and repeats that same command after ResolutionCommitUnknown. Applied reports the applied decision, and AlreadyApplied reports the original decision state without applying it again. Pruning a job removes its resolution receipts, so application evidence and business idempotency need their own retention.
- See public recovery consumer and operator contracts.

1.10 Runtime capacity and deadlines

Independent renewal protects lease progress from ordinary main-pool saturation. It does not eliminate infrastructure stalls.

Supervised runtime

responsibility

Own process and pool lifetimes, queue capacity, timers and shutdown.

interface

check/start/supervised/named/stop; queue tuning; without_consumers/without_pruner.

interactions

RestForOne supervision orders pool/deadline ownership, forwarder, runtime, per-queue consumers and pruner. Each consumer owns its reserved renewal pool.

invariants

Current schema is required before consumer start. Named runtime lookup survives restart. Failed startup unwinds all acquired resources, registry and caches. Running and ACK-pending attempts count against local capacity.

failure

Stop invalidates retained connections/lifetime tokens. Shutdown grace bounds cooperative drain; remaining workers leave fenced lease recovery. Node crash, database outage and OS suspension have no liveness guarantee.

- Queue defaults: ten local slots, 250-millisecond polling, thirty-second lease, fifteen-second shutdown grace. Queues are derived from registered workers; duplicate queue tuning and tuning unused queues are refused. Local slots do not cap all nodes or serialize equivalent job effects.

- Claim/refill is serialized one free slot per mailbox message. The coordinator yields between slots and owns at most one poll timer. Polling is the current wakeup mechanism; fairness, LISTEN/NOTIFY and live pause/scale are retained gaps.
- Storage deadline D defaults to four seconds. Lease L must satisfy L ≥ 4D. Renewal arms its L/3 timer before the query; the reserve isolates it from admission/ACK checkout. The timing argument assumes an established progressing connection, bounded storage work and actor progress; initial reconnect, same-row contention and OS scheduling remain outside it.

Boundary	Owned budget	Practical limit
Ordinary storage	Absolute deadline calculated before checkout; callback executes at most once	Queued pgo checkout has no unconditional receive timeout; late transferred candidates are refused before SQL
Migration step	Separate thirty-second default transaction budget	Large DDL and lock acquisition may fail; earlier steps remain committed
Borrowed transaction	Caller transaction timeout	No new Grind BEGIN/COMMIT or checkout budget
Handler	Fifteen-minute default execution timeout	Stopping a process cannot retract an already executed effect
await within	Monotonic wait checks after reads; adaptive polls	Final read may extend beyond within; pool queue wait may exceed D

- The FFI retires stale holders/reconnection candidates within one absolute deadline, probes sockets under a separate watchdog and preserves unrelated Erlang exceptions. A narrow pog conversion exception is mapped; broad catches never forge rollback. See ADR-0005, ADR-0008 and deadline boundary.

1.11 Reads, administration and retention

Typed read attachment checks stored contracts. Retention deliberately limits recovery.

Reads and administration

responsibility

Return committed typed outcomes and bounded payload-free operator views.

interface

bind/arguments/state/outcome/await; admin.list, admin.statistics, quarantine_expired, prune_finished and receipt reconciliation.

interactions

Reads check installation, queue, worker and codec versions. List uses ascending job ID, optional queue/state filters and a bounded page size.

invariants

Native values are decoded only by the retained compatible codecs. Uncertain is nonterminal. Administrative attribution is not access control.

failure

Missing job, stale installation, incompatible contract and invalid stored payload remain read errors. Timeout does not cancel work or prove absence of a commit.

- Outcome is Pending(state), Succeeded(output), Failed(failure, optional terminal cause, description), Discarded(reason), Cancelled(reason) or Uncertain(evidence). Business(error), BusinessUnrecorded and RuntimeFailure retain distinct causes. await starts at twenty milliseconds, doubles to a five-hundred-millisecond polling cap and reads committed state.

Queue statistics

responsibility

Return a content-free committed view of one explicitly selected queue.

interface

admin.statistics returns Statistics(sampled_at_ms, states), with one StateStatistics for each known state: count, oldest_job_age_ms, due_count and oldest_due_age_ms.

interactions

One statement uses one database snapshot and one clock sample through the same installation-scoped storage path as other administration reads. It reads committed rows without claiming, acknowledging, pruning or changing jobs.

invariants

All eleven states remain distinct and zero groups are present. Ages are optional exactly when their corresponding count is zero. Insertion age is time since admission, not time in the current state. Due includes only Queued, Scheduled and Retryable rows whose availability has arrived; it does not establish deployed compatibility, retry budget, capacity or authority to execute.

failure

A stopped runtime returns NotRunning; storage failure returns Unavailable; an unrecognized stored state returns RecordMismatch. No failure is reported as an empty queue. The existing checkout/deadline limitations remain.

- Job and due ages are elapsed milliseconds clamped at zero. The result contains no payloads, identifiers, keys, correlation or failure descriptions. Explicit queue selection bounds result cardinality, not storage work: cost grows with retained rows in that queue. State order is unspecified. Applications own authorization, export, sampling frequency and business-state interpretation; independently sampled installations or libraries do not form one atomic snapshot.

BEHAVIOR

Statistics isolate the selected queue

Given An installation contains independently named queues.

When The caller reads statistics for one queue.

Then Only that installation and exact queue name contribute jobs.

Then All eleven states are returned, including empty groups with zero counts and absent ages.

Then The read leaves jobs unchanged.

level: boundary area: Queue statistics

BEHAVIOR

Statistics observe one committed snapshot

Given Jobs can change state between separate administrative reads.

When The caller reads queue statistics.

Then All state groups describe one committed snapshot with one sampled database time.

Then The result excludes job contents and identifiers.

level: boundary area: Queue statistics

BEHAVIOR

Availability determines due statistics

Given A waiting job has a future availability time.

When The caller reads statistics after availability arrives.

Then The job contributes to the due count and due age in its retained Queued, Scheduled or Retryable state.

Then Reaching availability does not itself change the retained state or establish authority to execute.

level: boundary area: Queue statistics

BEHAVIOR

Statistics preserve outcome distinctions

Given Jobs retain completed outcomes, retry decisions, uncertain effects or contract refusals.

When The caller reads queue statistics.

Then Each job contributes to its own retained state group, including Uncertain and ContractMismatch.

Then Only waiting jobs whose availability has arrived contribute to due counts.

Then The result excludes failure descriptions and effect evidence.

level: boundary area: Queue statistics

BEHAVIOR

Executing jobs are counted without becoming due

Given A job is executing.

When The caller reads queue statistics.

Then The job contributes to Executing and does not contribute to a due count or due age.

level: boundary area: Queue statistics

BEHAVIOR

Statistics exclude staged admission

Given An admission is staged in a caller-owned transaction.

When The caller reads queue statistics before that transaction commits.

Then The staged job is absent.

Then A rollback leaves the job absent; a later committed admission is visible to a subsequent snapshot.

level: boundary area: Queue statistics

BEHAVIOR

Statistics refuse unavailable storage or runtime

Given The runtime is stopped or its storage cannot complete the read within the applicable storage deadline.

When The caller reads queue statistics.

Then A stopped runtime returns NotRunning; a storage failure returns Unavailable.

Then Neither failure is reported as an empty queue.

level: boundary area: Queue statistics

BEHAVIOR

A present age can be zero

Given The sampled database time precedes a retained job's insertion time.

When The caller reads statistics containing that job.

Then The job's insertion age is clamped to zero.

Then An age remains present when its corresponding count is nonzero.

level: boundary area: Queue statistics

BEHAVIOR

Statistics reject an unknown retained state

Given A job in the selected queue has an unrecognized retained state.

When The caller reads queue statistics.

Then The entire read returns RecordMismatch instead of omitting that job.

Then That job does not prevent a separate read of another queue.

level: boundary area: Queue statistics

Retention

responsibility

Delete old terminal jobs and dependent receipt rows in bounded concurrent-safe batches.

interface

Default pruner: seven-day retention, thirty-second interval, batch ten thousand; explicit admin.prune_finished.

interactions

Selects finished rows older than database-time cutoff using FOR UPDATE SKIP LOCKED, ordered by finished_at/ID; cascades receipt foreign keys. Admission candidate locks prevent deleting a selected retained conflict.

invariants

Only six terminal states with finished_at are pruned. Uncertain work remains retained. No leader election is needed for concurrent bounded pruning.

failure

Pruning permanently ends admission/ACK/resolution reconciliation and while_retained uniqueness. Runtime/schema outages can postpone deletion; configured retention is not an exact deletion instant.

- An application requiring permanent deduplication archives its own durable business identity. A Grind submission receipt is neither an eternal tombstone nor an effect ledger. See ADR-0007.
- The pruner first ticks thirty seconds after it starts and never drains a backlog in one tick; a batch of exactly the limit is taken again on the next tick. A concurrent pruner skips rows another holds, so every node can run one. Each prune call emits a completed observation with the deleted count or a failed observation with a coarse error class.
- A retention window shorter than a live lease can turn a late acknowledgement retry into QueueAckStale with AckRecordMissing. reconcile_acknowledgement returns ReceiptNotFound after pruning. A retry with job.with_id or job.unique under a pruned receipt is indistinguishable from a new request and inserts another job. A pending submission whose job was pruned before reconciliation never recovers its decision. An AllRetained or while_retained uniqueness key reopens once its occupying row is pruned, and a finite period needs a retention window at least as long as the period to keep its guarantee. A unique period measured from the schedule can outlive the finished-time retention of a cancelled job, and the two settings have no cross-check.
- The candidate scan orders by finished time and identifier through the finished index, and each receipt deletion seeks by job identifier through its own index. Retention is a property of the schema and never of one queue.

1.12 Observations

Durable rows establish outcomes; events explain transitions and runtime pressure.

Typed Sinal observations

responsibility

Publish payload-free lifecycle evidence and bounded operational diagnostics.

interface

Sixteen grind.telemetry descriptors, typed metadata codecs and measurement builders.

interactions

A supervised forwarder receives bounded producer messages and delivers through Sinal outside database callbacks.

invariants

Lifecycle events require commit reply or matching receipt. submit_in and pure reconcile_submission emit no admitted event; resolve_uncertain_in emits no resolved event. Diagnostics do not grant commit authority. Metadata excludes inputs/outputs, raw SQL/ settings/exception terms.

failure

Default forwarder capacity is 1024; events may drop or duplicate. Ordering is per producer only, monotonic time is local, subscriber failure cannot block queue correctness. Missing event does not imply missing job.

Family	Descriptors
Job lifecycle (8)	admitted, claimed, quarantined, acknowledged, resolved, cancellation_decided, released, contract_mismatch_recorded
Pruning (2)	completed, failed
Diagnostics (6)	renewal, acknowledgement, acknowledgement_retry, checkout, claim_failed, capacity

- Acknowledgement/resolution observations distinguish Replied and Reconciled and report the actual receipt disposition. A retry may publish another observation of the same commit. Consume stable identities when deduplicating operational counts.
- Renewal reports signed database lease headroom at measurement, not delivery. Checkout reports actual unclamped wait; nested storage wrappers do not double-count checkout. Capacity includes pending ACK occupancy. Native diagnostic enum codecs reject unknown stored labels.
- Handler cancellation observation requires progressing renewal and is best effort within its cadence, not a global delivery guarantee. See ADR-0009 and descriptor schemas.
- One bounded forwarder per runtime carries lifecycle, pruning and diagnostic events, sized by the observation capacity. A slow or raising handler stalls only the forwarder, never the coordinator that proved a commit. The forwarder is a temporary child nested under its own supervisor, so its termination never counts against the pool restart budget. When the nested restart budget is exhausted the forwarder stays unavailable for the runtime lifetime, and emission reports ForwarderUnavailable, which Grind discards while jobs continue normally.
- Delivery is best-effort. An event can arrive twice, for example a Reconciled observation after a Replied one with the same command ID, and can be lost to forwarder overflow, a forwarder restart or a commit-unknown outcome. Handlers run in the forwarder process, so the emitting process dictionary does not cross the hop. Durable truth is the tables and receipts: state, outcome and acknowledgement reconciliation answer any question that must not be lost or double-counted.
- A lifecycle event is emitted only once a commit is proven, never inside a transaction callback and never for a rolled-back, aborted, stale or commit-unknown outcome. The Replied confirmation proves the commit reply of that call, and the Reconciled confirmation proves a durable receipt read after a lost reply. Committed values come from what was committed, so a concurrent cancellation can override a non-uncertain proposal with a committed Cancelled. Retry-budget exhaustion is decided by the worker retry policy before acknowledgement, and the acknowledgement attempt-count check only rejects a stale acknowledgement.
- Receipt-read recovery calls never emit. When the originating call returned a commit-unknown outcome it emitted nothing, so the committed transition can stay unobserved even after reconciliation. This accepted gap makes durable reads the only complete account of a commit-unknown recovery.
- Cancellation decided before run or while running emits an event, and a repeated request against an executing job can emit again. AlreadyUncertain records intent without changing disposition and emits nothing, and AlreadyCancelled and AlreadyFinished are read-only and emit nothing. A released event reports a claimed attempt refunded before its worker ran. A contract_mismatch_recorded event reports a terminal nonclaimable state, and the job does not return to Queued.

Diagnostic	Reports
renewal	Renewed, skipped lock, unavailable live fence, storage failure or completion-renewal budget exhaustion, with optional signed lease headroom from the database clock
acknowledgement	Replied, reconciled, rolled back, unknown, rejected fence or command conflict
acknowledgement_retry	Retry number, delay and time pending; the handler is not rerun

Diagnostic	Reports
checkout	Actual pool wait, checkout candidates and full storage-call duration, labelled by operation and main or reserved pool
claim_failed	Quarantine-scan or candidate-claim failure with a sanitized error kind
capacity	Local maximum, active, handler-running, ACK-pending and available slots, and draining status

- Timing fields use microseconds, and lease headroom and retry delay use milliseconds. Renewal duration describes the batch call and repeats for each attempt it observed, so checkout events count storage calls. Capacity describes the coordinator local ledger and is not queue depth. Metadata carries node, consumer, queue and attempt identities and excludes payloads, error messages, SQL and connection settings.
- A missing live fence does not establish expiry, a skipped lock does not identify its owner, a completion-renewal budget exhaustion is a local limit and not proof of quarantine, and an unknown acknowledgement proves neither commit nor rollback. Checkout coverage includes quarantine scans, claims, batch renewal, acknowledgement transactions and their reconciliation reads, and excludes other public storage operations, proposal validation and contract-mismatch parking. Nested calls on a checked-out connection emit no further checkout event. Wait measures actual checkout calls including stale candidates, total duration also includes admission, SQL and cleanup, and a process killed before return can emit no sample.

1.13 Deployment obligations

The runtime contracts leave these decisions to the operating application.

- Operator commands require application-owned permissions, and attribution strings are not credentials. The client handle identity is a misuse check and not cross-cluster authentication. A caller-owned transaction is an open READ COMMITTED SingleConnection on the same database; a staged reply does not prove COMMIT, and a prior matching admission receipt does not prove that a retry business writes committed.
- After CommitUnknown the application retains the returned PendingSubmission and reconciles it, and it repeats a caller key only with the identical command. A missing receipt is unknown and not proof of rollback. Stable acknowledgement and resolution identities reconcile the original disposition. The application never repeats an external effect because a storage reply timed out, and it keeps a business ledger when permanent deduplication is required, because receipt recovery ends at pruning.
- Capacity planning counts one reserved renewal connection per used queue in addition to the main pool. Pending acknowledgements occupy local slots. The relation L at least 4D validates an established progressing reserved connection and does not guarantee liveness through reconnect, row contention or operating-system suspension. Queued checkout is not bounded by D and await within can exceed its bound by its final read, so long pool waits need external monitoring.
- Operators monitor signed lease headroom, actual checkout wait, acknowledgement retry phase and capacity including pending completion. Subscribers are never required for correctness, and diagnostic metadata never carries job payloads or raw SQL, settings or exception terms.
- The application uses cooperative stop and failed-start cleanup, and drains direct raw pog queries under its own ownership. A dependency change requires regression checks of the private driver topology and cache coupling. Runtime and pool names stay finite, because repeated consumer starts allocate atoms that OTP does not reclaim. Connection poolers and encrypted fault paths need explicit deployment qualification.

1.14 Verification and retained extensions

Independent consumers, PostgreSQL faults and a frozen oracle answer different questions.

Verification boundary

responsibility

Prove public typing, codec behavior, fenced PostgreSQL transitions and scoped comparison independently.

interface

testing.perform/perform_with/drain; consumer package; root tests; oracle manifests; benchmark and resilience harnesses.

interactions

Inline helpers round-trip input/output/configured error codecs without a database. Drain drives real manual attempts; oracle runs Oban separately against classified shared scenarios.

invariants

Inline helpers do not simulate lease, timeout, retry or persistence. External consumers cannot use internal modules. Passing a paired scenario proves only its declared equivalent configuration.

failure

Database-free helpers execute real application code in the calling process. Drain timeout kills its runner but leaves lease recovery for claimed work. Missing/invalid benchmark audit cannot count as a pass.

- testing.perform_with supplies synthetic Context overrides, including already-fired cancellation and an optional connection. Without an error codec, direct typed Failed(error) remains visible. drain handles one real job at a time up to its bound; disable automatic consumers to avoid competing processing.
- The primary behavioral oracle is Oban 2.24.1 open source. Grind default backoff, exact encoded uniqueness, opt-in matching, bounded attempt ownership and durable command receipts have deliberate differences. Pro families require original explicit contracts or a licensed oracle; source availability cannot be assumed.
- Root regression tests, separate consumer tests, migration conformance mirrors, fault proxies, scenario manifests, audit invariants and machine-readable catalogs are retained executable inputs. Measured summaries are ADR evidence and qualify only their recorded source and dependencies.

1.14.1 Complete capability inventory

Family	Current contract	Retained intended contract / missing surface
Job records and attempts	Typed input/output/error, correlation, eleven states and counters	Priority, tags, metadata, full attempt audit and suspension with explicit state/retention rules
Admission	Immediate/absolute/relative; receipts; borrowed transaction	Bulk commands with per-item atomicity, conflicts and ambiguity; uniqueness bulk needs independent contract
Queue control	Static local concurrency/poll/lease/grace	Pause/resume/scale and registration management; cluster/global/rate/partition limits with durable authority and fairness
Uniqueness	Exact key profile, incoming state/time filter, scheduled reschedule	Cross-worker domains, arbitrary states, broader replacements, key backfill and bulk selection
Schedules	One-off database availability	Cron/interval/timezones, DST, missed runs, clock jumps, durable tick identity and elected schedule ownership
Infrastructure/plugins	Concurrent pruner, polling and expiry quarantine	DynamicCron, Lifeline, Stager, Reindexer, leader/notifier contracts, PostgreSQL LISTEN/NOTIFY with loss/poll fallback
Dependency workflows	Ordinary jobs compose in application code	Named durable DAG steps, dependency values/codecs, waits and ignored cancelled/discarded upstream rules; distinct from Saga compensation
Batches	No batch coordinator	Membership and completed/attempted/discarded/cancelled callbacks; callback admission deduplication without exactly-once effect claim
Chunks	One input per handler	Homogeneous count/time flush, retained item IDs, one result per item, missing/duplicate result refusal and partial-failure accounting
Recorder and relay	Committed typed job output and bounded polling await	Named step result retrieval, reusable composition, notification optimization and recovery semantics beyond single-job outputs
Testing	Explicit inline helpers and manual drain	Typed enqueue inspection and broader modes without silently changing runtime authority

Family	Current contract	Retained intended contract / missing surface
Engine and storage	PostgreSQL implementation only	SQLite/engine adapter only after a real consumer establishes minimum port and isolation/clock/fencing semantics
Management	Payload-free list, resolution, prune/quarantine and sixteen events	Richer read/filter APIs, per-node/queue/plugin metrics and explicit operational permissions

- A local Saga inside a job may restart as a whole job under a declared replay-safe policy; it has no durable intermediate compensation journal. Durable Saga uses its own operation identities, codecs and storage, even when sharing Grind's pool. Fabric and Relay compose typed boundaries without a universal retry/error/runtime abstraction.
- See ADR-0001, ADR-0009, ADR-0010, ADR-0018, ADR-0019, source coverage, the oracle reference, the benchmark reference and oracle provenance. The qualification and acceptance rules are in Verification method.

1.15 Verification method

Each verification family states the question it answers and the limits of its evidence; the commands live in AGENTS.md.

1.15.1 Validation profiles

- The fast profile runs the edit-loop checks, and the full profile runs every applicable obligation before integration. Neither profile repairs source files. CI_ARTIFACT_DIR selects a fresh evidence parent, and the default is an ignored results directory per run.

Obligation	Check	Profile and enforcement	Evidence
Canonical formatting	treefmt through the flake check	Full CI, mechanism	Formatter diagnostics
Authored code compiles without warnings	All four Gleam projects, Erlang with warnings as errors, the oracle build and script compiler diagnostics	Fast and full, mechanism	Compiler diagnostics and negative fixture tests
Workflow, shell and Python correctness	Pinned actionlint, ShellCheck and narrow Ruff rules	Fast and full, partial	Lint diagnostics; vendored oracle dependencies and generated results excluded
Job, migration, consumer and scoped oracle contracts	The PostgreSQL gate	Full, mechanism for executed contracts	Core pair provenance, outputs and PostgreSQL logs including early failures
Harness correctness and fault boundaries	Benchmark smoke, fourteen Grind fault cases, Oban fault runs and the independent comparison	Full, bounded mechanism	Audit inputs, VM logs, snapshots, scenario completeness and comparison
Renewal and load audit contracts	The matrix profile	Weekly or manual, bounded mechanism	Warm-up plus three repeats, CSV, raw and drain evidence, and provenance
Two-hour mixed-fault requirement	The soak profile	Weekly or manual, bounded mechanism	Complete short suite, resource assertions, all nine fault types twice, elapsed soak and provenance

- CI runs on pushes to the default branch, pull requests and manual runs. The final CI status requires full validation and the design gate, and the Sinal test suite stays required. Weekly qualification has a separate final status. No commercial provider and no existing database is used, and fork pull requests run the same required checks.
- The Sinal sibling is checked out at the immutable revision in sibling-revisions.txt, using the default token to read the public repository and persisting no credentials. CI caches Gleam package archives and compiler-produced path dependency fingerprints, while the PostgreSQL and benchmark gates discard downloaded sources and compiled output before rebuilding. Dependency preparation retries explicit Hex rate limits at most three times, and compiler and test failures are never retried.

- The native script check compiles top-level Elixir modules and wraps remaining script expressions in a function without executing database or process actions, and it rejects compiler warnings and syntax errors. Source digests exclude credential paths before reading source bytes. Native gradual Elixir type checking is unavailable on the retained Elixir toolchain.

1.15.2 Qualification evidence

- A recorded result describes its source and dependencies only. A candidate record states the source and resolved dependency identities, cleanliness and content hashes, hardware and toolchain, exact workload, pool, concurrency and resource shape, delay semantics, warm-up and measured repeats, budgets, command exits, required markers, validity verdicts and failed-run classifications. A new dependency needs evidence for the behavior it affects, and a clean root tree does not prove a sibling path dependency was clean or built as recorded.
- Harnesses run serially in a fresh checkout, create disposable PostgreSQL clusters and write ignored results, and the host stays awake for long runs. The endurance requirement is two hours of mixed faults after warm-up plus fourteen standalone cases; a shorter rehearsal checks only the harness, and a twenty-four-hour run is not required.
- Acceptance requires every expected marker and paired case, and no skipped database suite counts. Claims, effects, receipts and authorized replay agree. Healthy siblings survive the chosen slow-acknowledgement and pressure configurations. T1 reports t1_triggered false with positive minimum headroom, because the runner exit alone does not enforce the lag verdict. T2 thresholds, arrival validity and resource checks keep their meaning. T3 crossing its seventy-percent trigger is a deferred optimization signal and not a correctness failure. Raw output is retained and reviewed independently through acceptance, and failed runs are recorded as failed before temporary output is deleted.
- Package review exports the hex tarball and builds the documentation locally, then builds a separate consumer from the export without sibling checkouts. Publication, tagging and pushing are separate owner actions. Local and CI dependency resolution align, and the accepted commit is recorded instead of inferred from historical counts.
- Deployment evidence stays bounded: laptop measurements establish no encrypted network faults, connection pooler behavior or arbitrary outage. The alternate OTP socket backend is unsupported by the pinned driver probe. Finite-name reuse and long consumer churn need bounded atom and resource review. The upstream rollback-after-crash failure can bypass a typed CommitUnknown through a process crash, and lease recovery stays authoritative.

1.15.3 Benchmark method

- The benchmark project measures Grind against disposable PostgreSQL clusters across drain throughput, empty-poll cost, open-loop latency, unique admission, pruning, renewal lag, slow acknowledgements and coordinator throughput. Each main point discards one warm-up and measures at least three repeats, and a subset establishes only its selected coverage. The suites, run commands, environment variables and retained results are in the benchmark reference.
- Compared runs match job count, handler cost, arrival pattern, concurrency, queue count, storage deadlines, leases, database settings and transport delay, and record machine, operating system, storage and toolchain. Every connection is counted, including each consumer's reserved renewal connection, and results are never portable capacity guarantees. Observer overhead stays equal when Grind is compared with Oban.
- A healthy run reconciles submissions, effects, final states, outputs, receipts and observations, and a missing measurement, an observer failure, an unacknowledged observer stop or an unexplained database error invalidates the evidence. L3 stays valid only while its arrival generator keeps its slots, and failed, unfinished and undispatched work stays in the denominator.
- T1 requires t1_triggered false, positive minimum headroom and no negative samples. T2 requires every slow target to activate, every healthy sibling to succeed with one matching receipt and no quarantine, and sampled headroom at least L divided by ten, and no acceptance run allows T2 failure. T3 crossing its seventy-percent trigger is a deferred optimization signal and not a correctness failure.
- Bookkeeping uses a harness database path and not Grind, and the failure that motivated this isolation is recorded in ADR-0009. The drain timeout is a soft polling budget and neither SQL cancellation nor the throughput denominator, and a timeout still fails.
- Each matrix writes a fresh output directory and rejects an existing one. Generated output is ignored and disposable after review, accepted findings are recorded concisely with provenance, and a clean run is a prerequisite and not a release verdict.

1.15.4 Resilience harness

- The resilience project launches independent named BEAM operating-system processes against one disposable PostgreSQL cluster and consumes the public API. Python controls processes and real TCP streams, so node identity is never simulated inside one VM, and a failed assertion fails the run. A worker appends and fsyncs each synthetic effect before its file barrier, and the controller requires an acknowledged effect_synced flag before a destructive fault. Only a persisted terminal state and receipt count establish durable completion.

ID	Controlled boundary	Required observation
M1	Two VMs consume one queue with two slots each	Both execute, local concurrency stays at or below two, and each effect and receipt occurs once
M2	SIGKILL a VM after its fsynced effect and before callback return	The survivor quarantines the job, no implicit replay occurs, and public AuthorizeReplay writes an attributed row and permits exactly one more effect
M3	Request-only, reply-only and bidirectional partitions of one VM	The healthy sibling completes, expired work becomes uncertain, and after healing the old callback returns and no stale receipt commits
M4	Forward COMMIT, suppress its reply, pause and kill the VM	A separate connection sees the committed receipt, a fresh VM resolves it by reconciliation, and no effect repeats
M5	Version 1 work stays active while a version 2 worker and codec VM joins	Version 2 work completes on its own node, killing version 1 leaves a row the version 2 quarantine scan still sees, and no implicit replay occurs
M6	Two VMs prune concurrently while another connection locks a terminal row	Both prunes finish, the total equals the unlocked candidate count, and a later sweep removes the locked row and its receipt
M7	Graceful drain, grace-expired shutdown and replacement with more capacity	Clean drain finishes durably, forced shutdown reports active work, and the replacement quarantines the abandoned attempt
F1	A PostgreSQL trigger delays acknowledgement updates while a long sibling renews, with 5 millisecond per-direction TCP delay	The database and the proxy witness the delay and all jobs reach durable success
F2	Close proxied connections and terminate real PostgreSQL backends	Cuts and terminations are recorded and the long job survives reconnect across a lease interval without another effect
F3	Kill the application worker process in a live VM after its effect	The M2 quarantine and audited-replay assertions hold
F4	Stop the disposable PostgreSQL server and restart it during a live attempt	Downtime is recorded and the job completes once after reconnect
F5	Ten consumer start and stop cycles beside a live sibling	Deadline, type and query cache counts return to baseline, the owner mailbox stays empty and sibling jobs keep completing
Soak	Repeated normal, retry and snooze batches, cancellations, lifecycle churn, pruning and nine destructive fault types	Exact attempt, effect and receipt accounting, every fault at least twice, a surviving long primary job and the resource bounds in every round

- Partition modes preserve the reliable byte stream: each direction retains at most one 64 KiB read, stops reading until recovery and forwards retained bytes before later bytes. Only the M4 reply-loss fault discards bytes. M6 covers concurrent maintenance because Grind has no leader election. The default suite uses a 1500 millisecond deadline and a 12000 millisecond lease and takes about two to four minutes; the real profile uses 4000 and 30000 and takes longer. Every soak runs all nine fault types at least twice, even when shorter than requested.
- Each run retains the source commit, dirty flag, source and dependency digest, frozen compiled runtime snapshot and binary hashes, scenario parameters, OTP and PostgreSQL versions, every BEAM process identifier and node name, fsynced effects, fault activation witnesses, held and released byte totals, COMMIT visibility, death confirmation, complete job, receipt and resolution rows, and resource samples. Dirty runs are labeled exploratory.
- After three warm-up batches, one recovered backend loss, one server restart and two initial samples, every drained round applies these bounds to both long-lived VMs. Process count is at most baseline plus four and total memory at most baseline plus 64 MiB. Deadline and type-cache entries equal the baseline, query-cache entries are at most baseline plus four and worker ETS entries are zero after drain. The owner mailbox is empty and total mailbox length is at most baseline plus 32. Atoms are at most baseline plus three per consumer start and 64 warm-up atoms, a declared lifecycle cost that does not claim reclamation. Database connections are at most the drained baseline plus two, retained primary schema storage is at most 16 MiB after prune and vacuum, and job and receipt counts are zero after each prune.

- A release-evidence run requires a clean Grind checkout, every short scenario and a soak of at least 7200 seconds, and the Sinal dependency is verified separately. TLS and pooler coverage remain separate, and the harness never claims exactly-once effects after an operator authorizes replay.

1.15.5 Oracle method

- The oracle pins Oban OSS 2.24.1 at one commit with its basic engine, claims no Oban Pro equivalence and preserves its license notice. The pin, the compared scenarios, the classification vocabulary, the deliberate differences, the run commands and the pin-upgrade procedure are in the oracle reference.
- A versioned scenario catalog drives a Grind adapter and an Oban adapter through public APIs. The comparator requires each executable scenario exactly once in each result stream under one catalog version and run identity, matches each result to its declared expectation and requires equivalent cases to match each other, so equal but wrong outputs fail. Completion is established from a committed row and never from handler invocation counts.
- Each catalog entry has one classification: equivalent, intentional-grind-semantics, missing-accidental-divergence or deferred. Differences are asserted separately and never normalized away, gap entries never count toward paired passes, and the Grind no-automatic-replay policy, live-lease fencing, durable receipts and audited replay are deliberate contracts that an Oban comparison never weakens.
- A run directory holds both result streams, the exact catalog snapshot and a provenance file with source, dependency, catalog, output and toolchain hashes. Existing result directories are refused, a source change during the pair fails the evidence check, a modified Oban checkout is rejected even when its commit matches, and dirty Grind source is labeled exploratory.
- The independent-VM fault fixtures specify M2 and M6 for Oban and Grind separately, with each node a distinct named BEAM process, and the comparator fails on missing intermediate evidence. They record two runs of one scenario contract with intentional differences and not equal runtime behavior. The ledger check verifies every local test name, upstream source path, catalog path and the Oban commit without a database. Correctness thresholds are exact, and the deterministic suite does not replace fault-proxy, multi-node or soak evidence.

1.15.6 Usage examples

- Usage is documented as compiled, runnable examples under examples/usage, one module per topic: limits and defaults, workers and codecs, jobs, transactions, runtime, handler context, handler testing, schema isolation, observations, uniqueness, uncertain jobs, retention and statistics, and migrations including cigogne. The README mirrors them in a short section and links to them, and no prose usage guide exists.
- The fast profile builds the examples with warnings as errors and runs the topics that need no database. The database gate runs every topic against disposable databases and fails unless each topic reached its last assertion, so an API change that breaks an example fails the gate. The examples consume Grind only through its public modules.

Glossary

Every term the layer declares, with the context that owns it.

Abandonment policy

owned by root

The worker policy deciding what follows an attempt whose handler timed out or whose node died: hold the job uncertain, or replay after lease expiry up to a bounded count.

Acknowledgement

owned by root

A retained proposal for a claimed attempt and a stable command identity. Its receipt records the actual committed disposition, which can differ from the proposal.

Admission receipt

owned by root

A durable record of a submission command fingerprint and its original Inserted, Existing or Rescheduled decision. Its lifetime ends when the referenced job is pruned.

Attempt fence

owned by root

The job, attempt identity, epoch, owner and live lease checked atomically before execution writes. Same-owner renewal preserves the fence; takeover invalidates it.

Attempt ordinal

owned by root

The retry-budget counter. Snooze and authorized abandonment replay refund it, so it can repeat and cannot fence ownership.

Available time

owned by root

The absolute database availability of a job. Relative admission delays are resolved using the database clock; recurring timezone schedules are a separate capability.

Borrowed transaction

owned by root

A caller-owned open READ COMMITTED PostgreSQL transaction used for admission or operator-resolution statements without Grind issuing BEGIN or COMMIT.

Cancellation intent

owned by root

A committed request to stop further work. It asks an executing handler to stop cooperatively and forbids replay of an uncertain job. It cannot undo an effect and does not prove that the handler stopped.

Capacity

owned by root

The per-consumer local slot budget occupied by running attempts and attempts with pending acknowledgements. It is not a cluster-wide execution limit.

Claim

owned by root

Database authority to execute an eligible job while its executing state, attempt identity, epoch, owner and lease remain current.

Codec

owned by root

A versioned typed JSON encoder and decoder. Encoder refusal is a value; semantic compatibility across versions belongs to the application.

Correlation

owned by root

Application-supplied or generated tracing context. It is neither the admission idempotency key nor an authorization credential.

Installation

owned by root

One Grind schema in a PostgreSQL database. Database privileges and schema separation provide isolation; the client identity token only guards accidental handle reuse.

Job

owned by root

A retained PostgreSQL record of one versioned worker input, its scheduling and accounting, and its committed outcome. Its integer identity is distinct from the caller submission key.

Key contract

owned by root

Full encoded input with its input codec version, or a named/versioned typed projection. Only jobs admitted with matching retained material participate.

Lease

owned by root

A database-time expiry for a claim. An expired lease grants no new acknowledgement or renewal write, even when the old process is alive.

Migration mirror

owned by root

A cigogne-format file under the migration directory holding the exact statements of one source migration version. Grind never reads it at runtime.

Migration shape

owned by root

The declared complete cumulative set of grind-prefixed relations and foreign-key constraints that a schema version contains. The schema probe compares it exactly with the actual schema.

Observation

owned by root

A best-effort typed Sinal event. Lifecycle observations follow evidence of a committed transition; diagnostic observations describe runtime behavior.

Operator resolution

owned by root

An attributed, idempotent command confirming success/business failure or authorizing replay of an uncertain job after application investigation.

Oracle

owned by root

A pinned external implementation whose observable behavior scoped scenarios are compared against. Oban OSS 2.24.1 is the primary oracle.

Avoid: reference implementation — Grind does not follow the comparator where it deliberately differs.

Oracle classification

owned by root

The label on each scenario catalog entry: equivalent, intentional-grind-semantics, missing-accidental-divergence or deferred. Only equivalent entries count as paired passes.

Pending submission

owned by root

The retained prepared command and fingerprint returned with CommitUnknown. It is the recovery token for reconciliation and does not prove admission.

Avoid: handle — a job handle exists only for a decided admission.

Qualification evidence

owned by root

Reviewed run output, with provenance, for one fixed source and dependency set. It qualifies only that build.

Avoid: benchmark result — a benchmark result lacks the validity verdicts and failed-run record.

Queue statistics

owned by root

A content-free committed snapshot of one selected queue: per state, a count, due count and oldest ages, read with one database clock sample.

Reserved renewal pool

owned by root

A one-connection pool owned by one used queue and dedicated to lease renewal, so ordinary main-pool saturation cannot starve renewal.

Resolution identity

owned by root

The installation-wide ID naming one exact operator resolution command: its decision, attribution and details. Reusing it for any different command is a conflict.

Retention

owned by root

The period after terminal finished_at during which jobs and their dependent receipts remain recoverable. Uncertain jobs have no finished_at and are excluded.

Snooze

owned by root

A handler response that reschedules a job without consuming a business attempt. A separate count bounds it, and exhausting that count is a business failure.

Staged result

owned by root

The reply of a call that runs inside a borrowed transaction. It states that statements executed in the caller transaction and does not establish commit.

Storage deadline

owned by root

The absolute deadline calculated before a storage call checks out its connection. It bounds callback execution and does not bound queued checkout wait.

Avoid: timeout — a timeout suggests every wait is bounded.

Submission key

owned by root

A caller-supplied or generated string identifying an admission command. Retrying the same retained request and key converges on its durable receipt.

Uncertain effect

owned by root

Evidence that an attempt may have performed an effect whose outcome is unknown. It is not evidence of failure or permission to retry.

Uniqueness domain

owned by root

A matching namespace including installation, worker identity/version, key contract and optional queue. Incoming state and time filters select candidates inside it.

Validation profile

owned by root

A named set of CI checks selected by one command: fast, full, matrix or soak.

Worker

owned by root

A deployed typed definition retaining worker identity, version, input/output/optional error codecs, queue, handler and execution policies.